

BASE DE DATOS 2

- **TÍTULO: TPI FOOD STORE – SISTEMA DE GESTIÓN DE PEDIDOS DE COMIDA**
- **INSTITUCIÓN: UTN – FACULTAD REGIONAL MENDOZA**
- **CARRERA: TECNICATURA UNIVERSITARIA EN PROGRAMACION**
- **MATERIA: BASE DE DATOS 2**
- **INTEGRANTES: MARIANO CHIRINO, FACUNDO QUIROGA , ANDRES FABRE.**
- **FECHA DE ENTREGA: JUEVES, 27 DE AGOSTO DE 2026,**

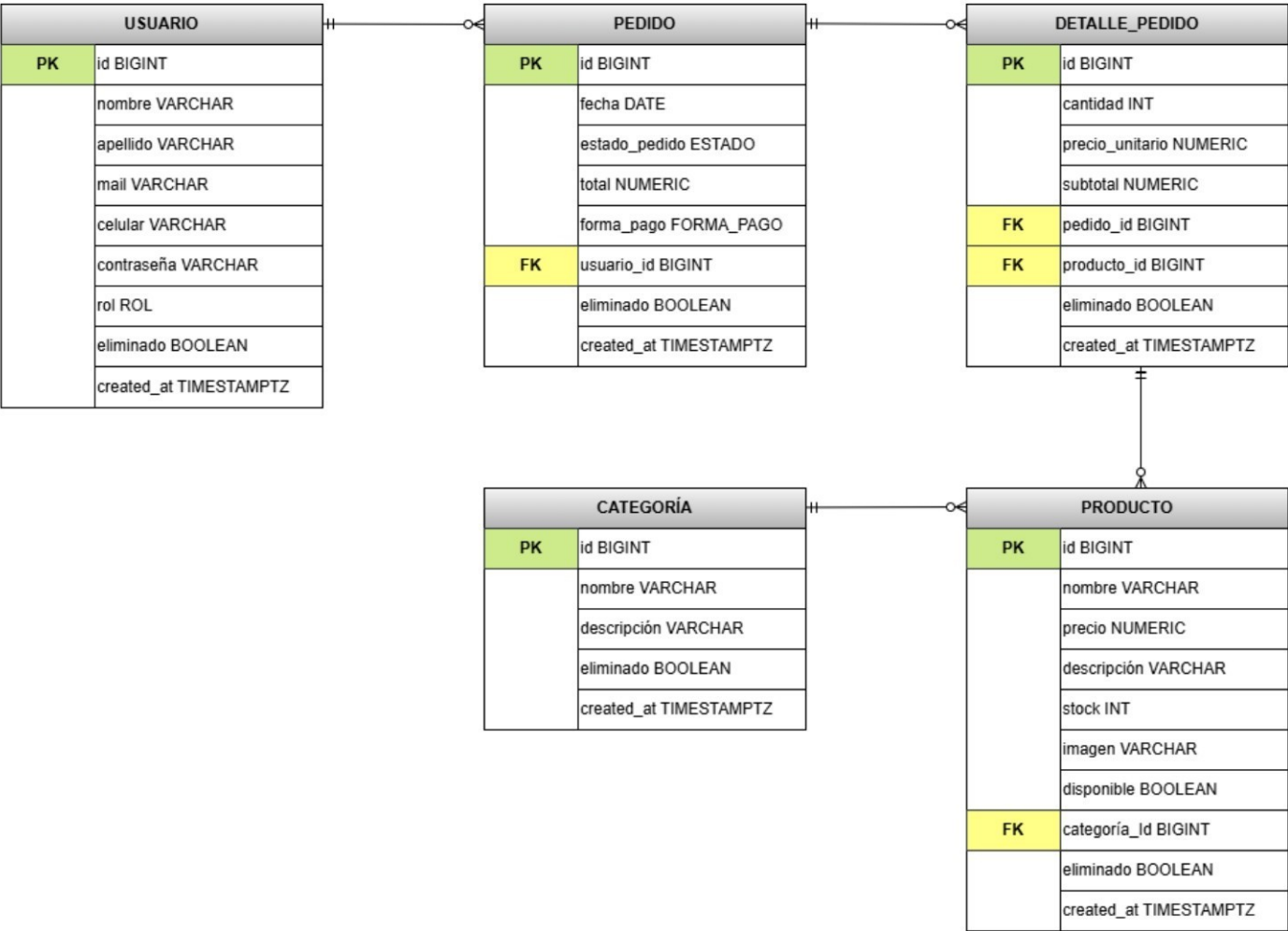
Índice

1.	Marco Teórico	2
2.	Modelo Conceptual (Entidad-Relación)	3
3.	¿Por qué el modelo tiene estas 5 tablas?	4
4.	Normalización — 1FN, 2FN y BCNF	5
5.	Justificación de Normalización — Atributos derivados	7
6.	Modelo Físico — DDL	8
7.	Índices y justificación	8
8.	Corrección detectada en el DDL de referencia	14
9.	Reglas de Negocio	14
10.	Lógica de Servidor — Vistas obligatorias	15
11.	Función de cálculo de total	16
12.	Triggers: subtotal y total automáticos	16
13.	Procedimiento transaccional: sp_crear_pedido	18
14.	Transacciones y Concurrencia	18
15.	Escenarios de transacciones y concurrencia	18
16.	Dificultades y soluciones	22
17.	Bibliografía / Webgrafía	24

Marco Teórico

El presente trabajo se sustenta en los conceptos fundamentales del modelo relacional de bases de datos. Un modelo relacional organiza la información en tablas (relaciones), donde cada fila representa una instancia de una entidad y cada columna un atributo de esa entidad, vinculando tablas entre sí mediante claves primarias (PK) y claves foráneas (FK) que garantizan la integridad referencial. La normalización es el proceso mediante el cual se estructura un esquema para eliminar redundancias y anomalías de actualización, inserción y borrado, verificando progresivamente el cumplimiento de la Primera (1FN), Segunda (2FN) y Tercera Forma Normal (3FN), así como la Forma Normal de Boyce-Codd (BCNF); en ocasiones, y de forma justificada, se opta por una denormalización controlada cuando el beneficio en rendimiento supera el costo de mantener cierta redundancia. La integridad de los datos se garantiza mediante restricciones declarativas (PRIMARY KEY, FOREIGN KEY, UNIQUE, CHECK, NOT NULL) y, cuando la lógica lo requiere, mediante triggers y procedimientos almacenados que ejecutan reglas de negocio directamente en el motor de base de datos. Finalmente, el control transaccional —mediante las propiedades ACID (Atomicidad, Consistencia, Aislamiento y Durabilidad)— asegura que operaciones compuestas por múltiples sentencias se completen en su totalidad o no se apliquen en absoluto (rollback), y los niveles de aislamiento (como READ COMMITTED o SERIALIZABLE) determinan cómo se comportan las transacciones concurrentes frente a lecturas y escrituras simultáneas sobre los mismos datos.

Modelo Conceptual (Entidad-Relación)



¿Por qué el modelo tiene estas 5 tablas?

El Food Store fue organizado en estas 5 tablas para reflejar adecuadamente los procesos de un negocio de comidas y cumplir con los principios de normalización hasta 3FN.

La tabla “categoría” permite organizar el menú y agrupar los productos por familias. Por otro lado, la tabla “producto” almacena el catálogo de ítems disponibles con su precio actual y stock disponible y mantiene una relación N:1 con “categoría”.

La tabla “usuario” concentra los datos de clientes y administradores con credenciales e información de contacto mientras que la tabla “pedido” modela la cabecera de la transacción de compra (cliente que la realizó, estado del pedido, forma de pago, fecha y el total acumulado) manteniendo una relación N:1 con “usuario”.

Finalmente, “detalle_pedido” resuelve la relación muchos a muchos (N:M) entre pedido y producto. Permite que un pedido contenga muchos productos y que un producto aparezca en diferentes pedidos. Además, es fundamental para congelar el precio_unitario cuando se produce una venta y asegurar que los cambios futuros en los precios no alteren el historial.

¿Por qué se repiten id/eliminado/created_at en todas las tablas?

Estos tres atributos se repiten en todas las entidades como una medida de repetición controlada. **id** (BIGINT GENERATED ALWAYS AS IDENTITY): Es una clave subrogada única y auto-incremental por tabla. Ofrece el máximo rendimiento para las consultas con JOIN y evita claves compuestas complejas.

eliminado (BOOLEAN DEFAULT FALSE): Implementa el mecanismo de borrado “soft delete”. Este mecanismo garantiza la integridad referencial histórica, si una categoría, producto o usuario se elimina sus datos no se borran de la base, previniendo la pérdida de historial.

created_at (TIMESTAMP TZ DEFAULT now()): Registra automáticamente el momento de creación del registro. Es indispensable para auditorías y análisis temporales.

Modelo Lógico

categoría(id, nombre, descripcion, eliminado, created_at)

producto(id, nombre, precio, descripcion, stock, imagen, disponible, categoria_id→categoria, eliminado, created_at)

usuario(id, nombre, apellido, mail, celular, contrasena, rol, eliminado, created_at)

pedido(id, fecha, estado, total, forma_pago, usuario_id→usuario, eliminado, created_at)

detalle_pedido(id, cantidad, precio_unitario, subtotal, pedido_id→pedido, producto_id→producto, eliminado, created_at)

Normalización — 1FN, 2FN y BCNF

Primera Forma Normal (1FN)

Un esquema está en 1FN cuando todos sus atributos son atómicos (no divisibles) y no existen grupos repetitivos dentro de una misma fila. Las cinco tablas del esquema cumplen esta condición: cada columna almacena un único valor indivisible (un VARCHAR, un NUMERIC, un BOOLEAN, etc.), y ninguna tabla guarda listas o conjuntos de valores en un solo campo.

El punto que exige justificación explícita es por qué los productos de un pedido no se guardan como una lista dentro de la fila de pedido (por ejemplo, un campo `productos` VARCHAR con ids separados por comas). Esa alternativa violaría 1FN directamente, porque el campo dejaría de ser atómico, y además haría imposible expresar restricciones básicas del negocio de forma declarativa: no se podría aplicar `UNIQUE(pedido_id, producto_id)` para evitar productos repetidos, ni un `CHECK(cantidad > 0)` por línea, ni construir un índice o un JOIN eficiente contra producto. Por eso el modelo resuelve la relación N:M entre `pedido` y `producto` con una tabla intermedia, `detalle_pedido`, donde cada fila representa exactamente una línea de un pedido — un valor atómico por celda, sin grupos repetitivos.

Segunda Forma Normal (2FN)

Un esquema en 1FN está en 2FN cuando, además, todo atributo no clave depende de la clave primaria completa — condición que solo es relevante cuando la clave primaria es compuesta.

En este modelo, las cinco tablas usan clave primaria simple y surrogada (`id BIGINT GENERATED ALWAYS AS IDENTITY`). Con una clave de un solo atributo, no existe la posibilidad de una dependencia parcial (dependencia de una parte de la clave y no de toda), por lo que la 2FN se cumple de forma trivial en las cinco tablas.

El caso interesante para discutir es `detalle_pedido`: si en lugar de un `id` surrogado se hubiera usado como clave primaria la combinación natural (`pedido_id, producto_id`), ahí sí correspondería analizar 2FN en serio. En ese escenario hipotético, atributos como `cantidad`, `precio_unitario` o `subtotal` seguirían dependiendo de la clave completa (no tiene sentido un `cantidad` que dependa solo de `pedido_id` o solo de `producto_id`), así que en este caso particular el esquema seguiría estando en 2FN aunque la clave fuera compuesta. Sin embargo, se prefirió la clave surrogada `id` porque simplifica las referencias desde otras tablas (aunque en este modelo nada referencia a `detalle_pedido`), mantiene consistencia de diseño con el resto del esquema, y evita tener que declarar `UNIQUE(pedido_id, producto_id)` como restricción aparte en lugar de que sea la clave misma — de todas formas, la restricción `UNIQUE(pedido_id, producto_id)` se mantuvo explícita en el DDL para preservar la regla de negocio de "no repetir producto en el mismo pedido".

Forma Normal de Boyce-Codd (BCNF)

Un esquema está en BCNF cuando, para toda dependencia funcional no trivial $X \rightarrow Y$, X es una superclave. Es una condición más estricta que 3FN porque contempla también las dependencias funcionales entre atributos que no son la clave primaria, cuando esos atributos forman una clave candidata alternativa.

La tabla donde esto es relevante es `usuario`, que tiene dos claves candidatas: `id` (clave primaria) y `mail` (por la restricción `UNIQUE (mail)`). Hay que verificar que ninguna dependencia funcional tenga como determinante algo que no sea una de estas dos claves candidatas:

- $id \rightarrow \{\text{nombre, apellido, mail, celular, contrasena, rol, eliminado, created_at}\}$ — determinante es superclave (PK). ✓
- $mail \rightarrow \{\text{id, nombre, apellido, celular, contrasena, rol, eliminado, created_at}\}$ — determinante es superclave (clave candidata por `UNIQUE`). ✓
- No existe ninguna dependencia funcional adicional entre atributos no-clave de `usuario` (por ejemplo, `nombre` no determina `apellido`, ni `celular` determina `rol`).

Las demás tablas (`categoria`, `producto`, `pedido`) tienen una única clave candidata (`id`), ya que sus columnas `UNIQUE` (como `categoria.nombre`) no generan dependencias funcionales adicionales relevantes fuera de la que ya cubre la PK. En `detalle_pedido`, el único punto de tensión con BCNF es la dependencia $id \rightarrow \{\text{cantidad, precio_unitario}\} \rightarrow \text{subtotal}$, que en rigor es una dependencia transitiva — pero esto ya fue identificado y tratado de forma explícita como una denormalización deliberada en la sección siguiente, no como un error de diseño no advertido.

El esquema cumple 1FN y 2FN de forma estricta en las cinco tablas. Respecto de BCNF, se cumple en `categoria`, `producto`, `pedido` y `usuario` (verificando explícitamente las dos claves candidatas de esta última). La única excepción documentada es la dependencia transitiva de `subtotal` (y, de forma análoga a nivel inter-tabla, de `total`), que se decidió mantener como denormalización controlada por las razones de rendimiento e historial congelado desarrolladas a continuación.

Justificación de Normalización — Atributos derivados (subtotal y total)

Análisis de dependencias funcionales

El esquema presenta dos atributos calculables a partir de otros: `subtotal` en `detalle_pedido` ($= \text{cantidad} \times \text{precio_unitario}$) y `total` en `pedido` ($= \text{suma de los subtotales de sus detalles}$).

En `detalle_pedido`, esto constituye una dependencia funcional transitiva intra-relación:

$$id \rightarrow \{\text{cantidad, precio_unitario}\} \rightarrow \text{subtotal}$$

Es decir, `subtotal` depende de otros atributos no-clave de la misma tabla, no directamente de la clave primaria. En sentido estricto, esto es una violación de 3FN.

En pedido, el caso es distinto: total no depende de otro atributo no-clave

de la misma tabla, sino de una agregación de filas de OTRA tabla (``detalle_pedido``). No es una dependencia transitiva clásica intra-relación, pero sí constituye una redundancia inter-tabla: el mismo valor puede obtenerse recalculando desde otra fuente.

Decisión de diseño: materializar (denormalización controlada)

Se optó por **guardar ambos valores como columna**, en lugar de calcularlos al momento de la consulta con una vista o función, por dos motivos:

1. **Rendimiento en lectura.** Los listados de pedidos (``v_pedidos_resumen``) y los reportes analíticos (facturación por categoría, ranking de usuarios por gasto, etc.) se consultan con mucha más frecuencia de la que se modifican los detalles de un pedido. Recalcular la suma en cada lectura sería costoso a medida que crece el volumen de datos.
2. **Historial congelado.** `precio_unitario` se congela al momento de la venta y no seguirá el precio actual del producto si este cambia después. Por lo tanto, ``subtotal`` no es un valor "derivable del estado actual", sino un hecho histórico que debe preservarse tal como ocurrió al momento de la operación.

Cómo se garantiza la integridad del dato derivado

La estrategia elegida no delega en la aplicación cliente la responsabilidad de mantener estos valores sincronizados (lo cual sería propenso a errores), sino que la garantiza el propio motor de base de datos mediante triggers:

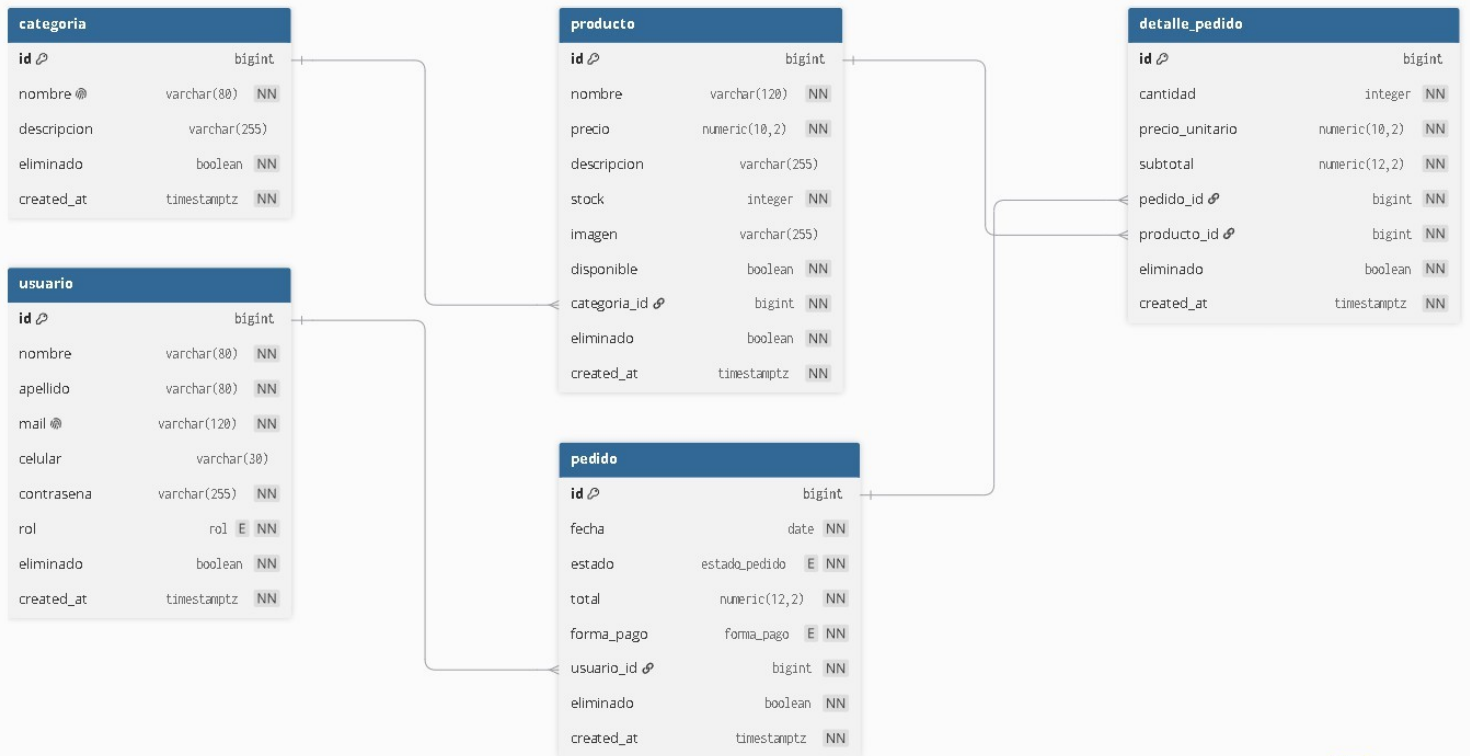
- **`fn_set_subtotal`** (BEFORE INSERT/UPDATE en ``detalle_pedido``): calcula ``subtotal`` automáticamente antes de guardar la fila, congelando el precio vigente del producto si no se especifica uno.
- **`fn_recalcular_total`** (AFTER INSERT/UPDATE en ``detalle_pedido``, FOR EACH STATEMENT, usando transition tables): recalcula ``pedido.total`` sumando los subtotales vigentes cada vez que cambian los detalles de un pedido.

De esta manera, ningún INSERT o UPDATE puede dejar ``subtotal`` o ``total`` desincronizados: el valor correcto se calcula siempre en el mismo momento en que se modifican los datos de los que depende, sin intervención manual.

Conclusión

El esquema no está en 3FN pura respecto de ``subtotal`` y ``total``, pero esta denormalización es deliberada, está documentada, y su consistencia está garantizada a nivel de motor (no de aplicación) mediante triggers — que es la práctica estándar en sistemas transaccionales donde el costo de recalculer en cada lectura supera el costo de mantener la redundancia controlada.

Modelo Físico — DDL



Índice 1 — idx_producto_categoria_id sobre producto(categoria_id)

Justificación: Soporta el listado y filtrado frecuente de productos pertenecientes a una categoría específica. Sin índice el planificador debe recorrer toda la tabla mediante un escaneo secuencial, leyendo 5.000 filas para encontrar los productos de la categoría especificada.

Por otro lado, con índice, el motor utiliza un Bitmap Index Scan / Index Scan, accediendo directamente a los bloques que contienen dicha categoría. Esto reduce el costo de ejecución e I/O en disco.

Sin índice:

Query

Query History

```

1  --Índice 1 - categoria_id en producto:*
2
3  DROP INDEX idx_producto_categoria_id;
4  EXPLAIN ANALYZE SELECT * FROM producto WHERE categoria_id = 1;
5  -- CAPTURA 1 (sin índice, debería decir "Seq Scan")
6

```

Data Output

Messages

Notifications

+

📄

▼

📋

▼

🗑️

📦

⬇️

📈

SQL

Showing rows: 1 to 8

🔍

📄

Page No:

QUERY PLAN	
text	
1	Seq Scan on producto (cost=0.00..124.50 rows=250 width=1081) (actual time=0.059..1.805 rows=250.00 loops=1)
2	Filter: (categoria_id = 1)
3	Rows Removed by Filter: 4750
4	Buffers: shared hit=62
5	Planning:
6	Buffers: shared hit=72 dirtied=2
7	Planning Time: 3.679 ms
8	Execution Time: 1.857 ms

Con Índice:

Query Query History

```

6
7 CREATE INDEX idx_producto_categoria_id ON producto(categoria_id);
8 EXPLAIN ANALYZE SELECT * FROM producto WHERE categoria_id = 1;
9 -- CAPTURA 2 (con índice, debería decir "Index Scan")

```

Data Output Messages Notifications

Showing rows: 1 to 12

QUERY PLAN	
text	
1	Bitmap Heap Scan on producto (cost=6.22..71.34 rows=250 width=1081) (actual time=0.618..0.768 rows=250.00 loops=1)
2	Recheck Cond: (categoria_id = 1)
3	Heap Blocks: exact=62
4	Buffers: shared hit=65 read=2
5	-> Bitmap Index Scan on idx_producto_categoria_id (cost=0.00..6.16 rows=250 width=0) (actual time=0.577..0.578 rows=...
6	Index Cond: (categoria_id = 1)
7	Index Searches: 1
8	Buffers: shared hit=3 read=2
9	Planning:
10	Buffers: shared hit=9 read=1
11	Planning Time: 49.895 ms
12	Execution Time: 4.604 ms

Índice 2 — idx_pedido_usuario_id sobre pedido(usuario_id)

Justificación: Optimiza las consultas del historial de pedidos realizadas por o para un usuario determinado. Sin índice el motor se ve obligado a realizar un Seq Scan evaluando la condición usuario_id = X sobre todos los registros de la tabla pedido.

Con índice el motor pasa a ejecutar un Bitmap Heap Scan / Index Scan, localizando los pedidos del usuario eficientemente

Sin índice:

Query Query History

```

12 --Índice 2 - usuario_id en pedido:*
13
14 DROP INDEX idx_pedido_usuario_id;
15 EXPLAIN ANALYZE SELECT * FROM pedido WHERE usuario_id = 1;
16 -- CAPTURA 3

```

Data Output Messages Notifications

Showing rows: 1 to 8

QUERY PLAN	
text	
1	Seq Scan on pedido (cost=0.00..44.00 rows=4 width=40) (actual time=0.217..0.556 rows=4.00 loops=1)
2	Filter: (usuario_id = 1)
3	Rows Removed by Filter: 1996
4	Buffers: shared hit=19
5	Planning:

Con índice:

Query
Query History

```

17
18 CREATE INDEX idx_pedido_usuario_id ON pedido(usuario_id);
19 EXPLAIN ANALYZE SELECT * FROM pedido WHERE usuario_id = 1;
20 -- CAPTURA 4

```

Data Output
Messages
Notifications

Showing rows: 1 to 12

QUERY PLAN	
	text
1	Bitmap Heap Scan on pedido (cost=4.31..14.85 rows=4 width=40) (actual time=0.120..0.126 rows=4.00 loops=1)
2	Recheck Cond: (usuario_id = 1)
3	Heap Blocks: exact=4
4	Buffers: shared hit=4 read=2
5	-> Bitmap Index Scan on idx_pedido_usuario_id (cost=0.00..4.31 rows=4 width=0) (actual time=0.091..0.091 rows=4.00 lo...
6	Index Cond: (usuario_id = 1)
7	Index Searches: 1
8	Buffers: shared read=2
9	Planning:
10	Buffers: shared hit=5 read=1
11	Planning Time: 2.602 ms
12	Execution Time: 0.160 ms

Índice 3 — Parcial idx_producto_nombre_vigente sobre producto(nombre) WHERE eliminado = FALSE

Justificación: Optimiza la búsqueda por nombre de productos en el catálogo. Al ser un índice parcial, ignora los registros donde eliminado = TRUE, lo que disminuye el tamaño del índice en disco y acelera las operaciones de mantenimiento como INSERT/UPDATE.

Sin índice (DROP INDEX): La búsqueda WHERE nombre = 'Producto 100' AND eliminado = FALSE requiere un Seq Scan completo pero con índice (CREATE INDEX): Se realiza un Index Scan directo, logrando un tiempo de ejecución prácticamente nulo.

Sin índice:

Query Query History

```

23 --Índice 3 - nombre en producto (parcial):*
24
25 DROP INDEX idx_producto_nombre_vigente;
26 EXPLAIN ANALYZE SELECT * FROM producto WHERE nombre = 'Producto 100' AND eliminado = FALSE;
27 -- CAPTURA 5
28

```

Data Output Messages Notifications

Showing rows: 1 to 8 Page No: 1

QUERY PLAN	
	text
1	Seq Scan on producto (cost=0.00..124.50 rows=1 width=1081) (actual time=0.064..1.521 rows=1.00 loops=1)
2	Filter: ((NOT eliminado) AND ((nombre)::text = 'Producto 100'::text))
3	Rows Removed by Filter: 4999
4	Buffers: shared hit=62
5	Planning:
6	Buffers: shared hit=17
7	Planning Time: 3.062 ms
8	Execution Time: 1.552 ms

Con índice:

Query Query History

```

30
31 CREATE INDEX idx_producto_nombre_vigente ON producto(nombre) WHERE eliminado = FALSE;
32 EXPLAIN ANALYZE SELECT * FROM producto WHERE nombre = 'Producto 100' AND eliminado = FALSE;
33 -- CAPTURA 6
34

```

Data Output Messages Notifications

Showing rows: 1 to 8 Page No: 1

QUERY PLAN	
	text
1	Index Scan using idx_producto_nombre_vigente on producto (cost=0.28..8.30 rows=1 width=1081) (actual time=0.086..0.087 rows=1.00 lo...
2	Index Cond: ((nombre)::text = 'Producto 100'::text)
3	Index Searches: 1
4	Buffers: shared hit=1 read=2
5	Planning:
6	Buffers: shared hit=10 read=1
7	Planning Time: 2.692 ms
8	Execution Time: 0.111 ms

Corrección detectada en el DDL de referencia

Durante la implementación se detectó un error tipográfico en el fragmento de DDL provisto por la cátedra: la definición de la clave primaria de categoría utilizaba el tipo BIGGINT (con doble "G"), que no es un tipo válido en PostgreSQL. Se corrigió a BIGINT, tipo correcto utilizado de forma consistente en el resto de las tablas.

Reglas de Negocio

Las siguientes reglas de negocio se implementan mediante restricciones declarativas (CHECK, UNIQUE, NOT NULL) cuando es posible, y mediante triggers cuando la regla no puede expresarse con una restricción simple:

Regla de negocio	Mecanismo en la BD
Precio y stock no negativos	CHECK (precio >= 0), CHECK (stock >= 0)
Cantidad de detalle > 0	CHECK (cantidad > 0)
Mail de usuario único	UNIQUE (mail)
No repetir producto en un mismo pedido	UNIQUE (pedido_id, producto_id)
Pedido siempre con usuario	usuario_id NOT NULL + FOREIGN KEY
subtotal = cantidad × precio_unitario (precio congelado)	Trigger BEFORE INSERT/UPDATE en detalle_pedido
total del pedido = Σ subtotales	Trigger AFTER ... FOR EACH STATEMENT en detalle_pedido
Soft delete (baja lógica)	Columna eliminado, nunca DELETE físico
No vender producto sin stock / no disponible	Validación en sp_crear_pedido con SELECT ... FOR UPDATE + descuento de stock

Data Output Messages Notifications			
			
			SQL
	id [PK] bigint 	nombre character varying (120) 	stock integer 
1	15	Producto temporalmente no disponible	10

Lógica de Servidor:

Vistas obligatorias

Se crearon las 4 vistas pedidas por la consigna. Todas filtran por eliminado = FALSE para respetar el borrado lógico.

v_categorias_vigentes — categorías activas:

```
CREATE VIEW v_categorias_vigentes AS
SELECT id, nombre, descripcion
FROM categoria
WHERE eliminado = FALSE;
```

v_productos_vigentes — catálogo activo, con el nombre de la categoría resuelto vía JOIN:

```
CREATE VIEW v_productos_vigentes AS
SELECT p.id, p.nombre, p.precio, p.stock,
       c.nombre AS categoria
FROM producto p
JOIN categoria c ON c.id = p.categoria_id
WHERE p.eliminado = FALSE AND c.eliminado = FALSE;
```

v_pedidos_resumen — cabecera de pedidos con el nombre completo del usuario:

```
CREATE VIEW v_pedidos_resumen AS
SELECT ped.id,
       u.nombre || ' ' || u.apellido AS usuario,
       ped.fecha, ped.estado, ped.forma_pago, ped.total
FROM pedido ped
JOIN usuario u ON u.id = ped.usuario_id
WHERE ped.eliminado = FALSE;
```

v_pedido_detalle — líneas de cada pedido con el nombre del producto:

```
CREATE VIEW v_pedido_detalle AS
SELECT dp.pedido_id,
       pr.nombre AS producto,
       dp.cantidad, dp.precio_unitario, dp.subtotal
FROM detalle_pedido dp
JOIN producto pr ON pr.id = dp.producto_id
WHERE dp.eliminado = FALSE;
```

Estas vistas son la base de varias historias de usuario (HU-CAT-01, HU-PROD-01, HU-PED-01) y evitan repetir los mismos JOIN y el mismo filtro de soft delete en cada consulta.

Función de cálculo de total

Se implementó como una función SQL simple (no PL/pgSQL, porque no necesita lógica de control) que suma los subtotales vigentes de un pedido. Es la que usa internamente el trigger de recálculo de total:

```
CREATE OR REPLACE FUNCTION calcular_total_pedido(p_pedido_id BIGINT)
RETURNS NUMERIC(12,2) AS $$
    SELECT COALESCE(SUM(subtotal), 0)
    FROM detalle_pedido
    WHERE pedido_id = p_pedido_id AND eliminado = FALSE;
$$ LANGUAGE sql STABLE;
```

Se marcó como STABLE (y no VOLATILE) porque, para los mismos parámetros y dentro de una misma sentencia, siempre devuelve el mismo resultado si los datos no cambian: esto le permite al planificador optimizarla cuando se la usa dentro de otras consultas.

Triggers: subtotal y total automáticos

La justificación de por qué se decidió materializar subtotal y total (en lugar de calcularlos solo en el momento de la consulta) ya se desarrolló en la sección de Normalización. Acá se documenta el código que garantiza esa consistencia.

Trigger BEFORE INSERT/UPDATE — congela el precio y calcula el subtotal de la línea:

```
CREATE OR REPLACE FUNCTION fn_set_subtotal()
RETURNS TRIGGER AS $$
BEGIN
    IF NEW.precio_unitario IS NULL THEN
        SELECT precio INTO NEW.precio_unitario
        FROM producto WHERE id = NEW.producto_id;
    END IF;

    NEW.subtotal := NEW.cantidad * NEW.precio_unitario;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_subtotal
BEFORE INSERT OR UPDATE ON detalle_pedido
FOR EACH ROW EXECUTE FUNCTION fn_set_subtotal();
```

Al ejecutarse BEFORE y FOR EACH ROW, modifica NEW antes de que la fila se escriba físicamente: si no se envía precio_unitario, lo toma del precio vigente del producto (lo "congela"); si se envía uno explícito, lo respeta. En cualquier caso, subtotal siempre queda calculado por el propio motor, nunca por el cliente.

Triggers AFTER INSERT/UPDATE — recalculan el total del pedido:

```
CREATE OR REPLACE FUNCTION fn_recalcular_total()
RETURNS TRIGGER AS $$
BEGIN
    UPDATE pedido p
    SET total = calcular_total_pedido(p.id)
    WHERE p.id IN (SELECT pedido_id FROM afectados);
    RETURN NULL;
END;
$$ LANGUAGE plpgsql;
```

```

CREATE TRIGGER trg_total_ins
AFTER INSERT ON detalle_pedido
REFERENCING NEW TABLE AS afectados
FOR EACH STATEMENT EXECUTE FUNCTION fn_recalcular_total();

CREATE TRIGGER trg_total_upd
AFTER UPDATE ON detalle_pedido
REFERENCING NEW TABLE AS afectados
FOR EACH STATEMENT EXECUTE FUNCTION fn_recalcular_total();

```

A diferencia del trigger de subtotal, este es FOR EACH STATEMENT (una sola ejecución por sentencia, no por fila) y usa una transition table (afectados) para acceder a todas las filas que cambiaron en esa sentencia de una sola vez. Esto evita recalcular el mismo pedido varias veces si una sola sentencia insertó o actualizó varios detalles del mismo pedido. Se necesitan dos triggers separados (uno para INSERT y otro para UPDATE) porque PostgreSQL no permite declarar una transition table para más de un evento en el mismo CREATE TRIGGER.

Procedimiento transaccional: sp_crear_pedido

Encapsula el alta de un pedido junto con todos sus detalles en una sola unidad atómica, replicando lo que en Programación 2 se resolvía con `connection.setAutoCommit(false)` y `rollback` manual:

```

CREATE OR REPLACE PROCEDURE sp_crear_pedido(
    p_usuario_id BIGINT,
    p_forma_pago forma_pago,
    p_items JSONB -- [{"producto_id":1,"cantidad":2}, ...]
) AS $$
DECLARE
    v_pedido_id    BIGINT;
    v_item          JSONB;
    v_producto_id  BIGINT;
    v_cantidad      INTEGER;
    v_stock         INTEGER;
    v_disponible    BOOLEAN;
BEGIN
    IF NOT EXISTS (SELECT 1 FROM usuario
                   WHERE id = p_usuario_id AND eliminado = FALSE) THEN
        RAISE EXCEPTION 'Usuario % inexistente o eliminado', p_usuario_id;
    END IF;

    INSERT INTO pedido(usuario_id, forma_pago)
    VALUES (p_usuario_id, p_forma_pago)
    RETURNING id INTO v_pedido_id;

    FOR v_item IN SELECT * FROM jsonb_array_elements(p_items) LOOP
        v_producto_id := (v_item->>'producto_id')::BIGINT;
        v_cantidad     := (v_item->>'cantidad')::INTEGER;

        SELECT stock, disponible INTO v_stock, v_disponible
        FROM producto WHERE id = v_producto_id AND eliminado = FALSE
        FOR UPDATE;

        IF NOT FOUND THEN
            RAISE EXCEPTION 'Producto % inexistente o eliminado', v_producto_id;
        END IF;
        IF NOT v_disponible THEN
            RAISE EXCEPTION 'Producto % no disponible', v_producto_id;
        END IF;
    END LOOP;
END;

```

```

    IF v_stock < v_cantidad THEN
        RAISE EXCEPTION 'Stock insuficiente (producto %): hay %, se piden %',
            v_producto_id, v_stock, v_cantidad;
    END IF;

    INSERT INTO detalle_pedido(pedido_id, producto_id, cantidad)
    VALUES (v_pedido_id, v_producto_id, v_cantidad);

    UPDATE producto SET stock = stock - v_cantidad WHERE id = v_producto_id;
END LOOP;
END;
$$ LANGUAGE plpgsql;

```

Dos decisiones de diseño importan acá:

1. `SELECT ... FOR UPDATE` bloquea la fila del producto antes de leer su stock. Esto serializa el acceso concurrente: si dos operadores intentan vender el mismo producto al mismo tiempo, el segundo espera a que el primero termine su transacción antes de leer el stock, evitando la sobreventa. Esto se demuestra en detalle en la sección Transacciones y Concurrency.

2. Un `PROCEDURE` en PostgreSQL no abre su propia transacción: se ejecuta dentro de la transacción del llamador (la que abre el `CALL`, o un `BEGIN...COMMIT` explícito). Por eso, si cualquier `RAISE EXCEPTION` se dispara — usuario inexistente, producto inexistente, sin stock — la excepción no capturada aborta toda la transacción envolvente: no queda ni el pedido ni ningún detalle insertado. El `rollback` no lo gestiona el procedimiento, sino que es consecuencia de la propagación de la excepción hacia la transacción que lo invocó.

La captura de pantalla del error "Stock insuficiente (producto 7): hay 98, se piden 9999" que se incluye en este informe corresponde justamente a este procedimiento rechazando una venta imposible antes de tocar la base — evidencia que se retoma como Escenario 1 (Atomicidad) en la siguiente sección.

Transacciones y Concurrency

Esta sección demuestra, con evidencia ejecutada sobre PostgreSQL 17 (pgAdmin 4), los cuatro puntos que exige la consigna: atomicidad, una transacción manual con `COMMIT` y otra con `ROLLBACK`, aislamiento entre transacciones concurrentes, y el efecto de los bloqueos con `SELECT ... FOR UPDATE`. El script completo (transacciones.sql) está en el repositorio; acá se explica qué se probó y qué se observó en cada caso.

Escenario 1 — Atomicidad

Se llamó a `sp_crear_pedido` con una cantidad que excede el stock disponible del producto id 7, para verificar que el procedimiento revierte todo el intento de compra ante un ítem inválido.

Resultado observado: el procedimiento lanzó `ERROR: Stock insuficiente (producto 7): hay 98, se piden 9999`, y no quedó registrado ni el pedido ni ningún detalle asociado. La excepción se propaga fuera de la función y aborta toda la transacción envolvente, tal como describe el diseño de `sp_crear_pedido`.

Data OutputMessagesNotifications

ERROR: Stock insuficiente (producto 7): hay 80, se piden 9999

CONTEXT: función PL/pgSQL sp_crear_pedido(bigint,forma_pago,jsonb) en la línea 38 en RAISE

SQL state: P0001

Total rows: 1

Query complete 00:00:00.062

CRLF

Ln 1, Col 1

Data OutputMessagesNotifications

Showing rows: 1 to 1

Page No: 1 of 1

pedidos
bigint

18

Data OutputMessagesNotifications

Showing rows: 1 to 1

Page No: 1 of 1

detalles
bigint

115

Total rows: 1

Query complete 00:00:00.127

CRLF

Ln 1, Col 1

Total rows: 1

Query complete 00:00:00.062

CRLF

Ln 1, Col 1

Escenario 2 — Transacción manual: BEGIN/COMMIT vs BEGIN/ROLLBACK

Se ejecutó un UPDATE que descuenta stock del producto id 7 dentro de un bloque BEGIN . . . COMMIT, y el mismo tipo de UPDATE dentro de un bloque BEGIN . . . ROLLBACK, verificando el stock antes y después de cada operación.

Resultado observado:

Paso	Stock	¿Persiste?
Valor inicial	94	—
Dentro de la transacción, antes de COMMIT	90	No todavía
Después de COMMIT	90	Sí
Dentro de la transacción, antes de ROLLBACK	89	No todavía
Después de ROLLBACK	90	No, se revirtió

El COMMIT hace permanente el cambio; el ROLLBACK lo deshace por completo, devolviendo el stock al valor que tenía antes de esa transacción — evidencia directa de atomicidad y consistencia.

Data Output		Messages	Notifications						
									SQL
	stock integer 								
1	94								
Total rows: 1		Query complete 00:00:00.298							

Data Output		Messages	Notifications
ROLLBACK			
Query returned successfully in 244 msec.			
Total rows:		Query complete 00:00:00.244	

Data Output		Messages	Notifications
≡+ 📄 ▼ 📋 ▼ 🗑️ 🗄️ ⬇️ 📈 SQL			
	stock integer 🔒		
1	90		
Total rows: 1		Query complete 00:00:00.236	

Escenario 3 — Aislamiento: lectura no repetible (READ COMMITTED vs SERIALIZABLE)

Se usaron dos sesiones concurrentes de Query Tool sobre el stock del producto id 7 (valor de partida: 90, luego de los ajustes del Escenario 2). La Sesión A abre una transacción y lee el stock dos veces; entre ambas lecturas, la Sesión B ejecuta un `UPDATE` que descuenta 5 unidades y confirma inmediatamente con `COMMIT`.

Bajo `READ COMMITTED` (nivel por defecto de PostgreSQL): la primera lectura de la Sesión A dio 90; la Sesión B confirmó su descuento (stock → 85); la segunda lectura de la Sesión A, ejecutada dentro de la misma transacción sin haber hecho ningún commit propio, dio 85 — reflejando el cambio ya confirmado por B. Es una lectura no repetible: dos `SELECT` idénticos, en la misma transacción, devolvieron valores distintos.

Bajo `SERIALIZABLE`: repitiendo la prueba con `SET TRANSACTION ISOLATION LEVEL SERIALIZABLE;`, la primera lectura de la Sesión A dio 85 (el stock ya actualizado por la corrida anterior); la Sesión B volvió a confirmar un descuento de 5 (stock → 80); pero la segunda lectura de la Sesión A siguió dando 85, a pesar de que B ya había confirmado su cambio. PostgreSQL le da a una transacción `SERIALIZABLE` una fotografía (snapshot) fija de los datos al momento del `BEGIN`, y esa transacción no ve cambios confirmados por otras sesiones hasta que ella misma termina. Al hacer `COMMIT` en la Sesión A, este se confirmó sin conflicto porque A solo había leído datos, sin escribir.

Data Output

Messages

Notifications

Escenario 4 — Bloqueos: SELECT ... FOR UPDATE evita la sobreventa

Se simuló el escenario de dos operadores intentando vender el producto id 3 (stock disponible: 15 unidades) casi al mismo tiempo, cada uno con un pedido de 8 unidades a través de `sp_crear_pedido`, que internamente usa `SELECT ... FOR UPDATE` antes de descontar stock.

Resultado observado: la Sesión A ejecutó el `CALL` con éxito y quedó con el lock de la fila del producto tomado, sin hacer `COMMIT` todavía. La Sesión B, al intentar comprar el mismo producto mientras A no había confirmado, quedó bloqueada esperando (mensaje *"Waiting for the query to complete..."* durante más de 30 segundos) — no leyó nada, simplemente esperó a que se liberara el lock. Apenas la Sesión A hizo `COMMIT`, se liberó el bloqueo, la Sesión B pudo leer el stock ya actualizado (7 unidades restantes) y el procedimiento respondió con `ERROR: Stock insuficiente (producto 3): hay 7, se piden 8: un rechazo de negocio prolijo y esperado, sin sobreventa`. El stock final quedó en 7, valor consistente — si no hubiera existido el bloqueo, ambas sesiones habrían leído 15 unidades disponibles al mismo tiempo y las dos hubieran descontado 8, dejando el stock en un valor negativo o inconsistente.

Data OutputMessagesNotifications

SQL

Showing rows: 1 to 1Page No: 1 of 1

	id [PK] bigint	stock integer
1	3	15

Data OutputMessagesNotifications	Data OutputMessagesNotifications
CALL Query returned successfully in 142 msec.	
Total rows: Query complete 00:00:00.142CRLF Ln 1, Col 1	Total rows: Waiting for the query to complete... 00:00:33.537CRLF Ln 1, Col 1

Data OutputMessagesNotifications	Data OutputMessagesNotifications
COMMIT Query returned successfully in 97 msec.	ERROR: Stock insuficiente (producto 3): hay 7, se piden 8 CONTEXT: función PL/pgSQL sp_crear_pedido(bigint,forma_pago,jsonb) en la línea 38 en RAISE SQL state: P0001
Total rows: Query complete 00:00:00.097CRLF Ln 3, Col 1	Total rows: Query complete 00:00:06.930CRLF Ln 1, Col 1

Data OutputMessagesNotifications

SQL

Showing rows: 1 to 1Page No: 1 of 1

	id [PK] bigint	stock integer
1	3	7

Total rows: 1

Query complete 00:00:00.086

CRLF

Ln 1, Col 1

Los cuatro escenarios muestran, con evidencia real y no solo teórica, que: (1) el procedimiento garantiza atomicidad por propagación de excepciones dentro de la transacción del llamador; (2) `COMMIT` y `ROLLBACK` producen estados finales distintos y verificables; (3) el nivel de aislamiento cambia lo que una transacción ve de los cambios de otras, siendo `SERIALIZABLE` más estricto que `READ COMMITTED`; y (4) `SELECT ... FOR UPDATE` es lo que efectivamente evita la sobreventa bajo concurrencia real, serializando el acceso a la fila en disputa en lugar de dejar que ambas transacciones decidan sobre datos desactualizados.

Dificultades y soluciones

Error tipográfico en el DDL de referencia. El fragmento de `schema.sql` provisto por la cátedra definía la clave primaria de `categoria` con el tipo `BIGGINT` (doble "G"), que no existe en PostgreSQL. Se detectó al intentar ejecutar el script y arrojar un error de sintaxis. Solución: se corrigió a `BIGINT`, manteniendo consistencia con el resto de las tablas.

Comentario faltante en `transacciones.sql`. Al armar el Escenario 4 (bloqueo con `FOR UPDATE`), el bloque de comentarios que separaba los escenarios quedó sin el prefijo `--` en una línea, lo que provocaba un error de sintaxis al ejecutar el script completo de una sola vez. Solución: se corrigió el comentario y se recomienda, para este tipo de scripts con escenarios de concurrencia, ejecutarlos línea por línea o bloque por bloque en vez de correr el archivo entero, ya que varios pasos requieren intervención manual entre dos sesiones simultáneas.

Interpretación de resultados en pgAdmin con múltiples sentencias. Al ejecutar varias sentencias SQL seleccionadas juntas (por ejemplo, `BEGIN; UPDATE; SELECT; COMMIT;`), la pestaña "Data Output" de pgAdmin solo muestra el resultado de la última sentencia ejecutada, ocultando los `SELECT` intermedios. Esto generó confusión al capturar evidencia de los escenarios de transacciones. Solución: se ejecutaron los bloques de a una o dos sentencias por vez, dejando el `SELECT` de verificación como última instrucción de cada selección, para poder capturar cada estado intermedio de forma clara.

Demostrar el bloqueo de `FOR UPDATE` en un entorno de una sola máquina. Simular dos operadores concurrentes reales requirió abrir dos pestañas de Query Tool independientes contra la misma base en pgAdmin, coordinando manualmente el orden de ejecución entre ambas sesiones (dejar la Sesión A sin `COMMIT` mientras se ejecutaba la Sesión B) para poder observar el estado de espera ("Waiting for the query to complete...") antes de liberar el lock.

Bibliografía / Webgrafía

- PostgreSQL Global Development Group. *PostgreSQL 16 Documentation*. <https://www.postgresql.org/docs/16/index.html> — Documentación oficial consultada para tipos de datos, `CREATE TABLE`, restricciones (`CHECK`, `UNIQUE`, `FOREIGN KEY`), tipos `ENUM` y `IDENTITY`.
- PostgreSQL Global Development Group. *Chapter 39. Triggers*. <https://www.postgresql.org/docs/16/triggers.html> — Referencia para el diseño de `trg_subtotal`, `trg_total_ins` y `trg_total_upd`, incluyendo el uso de transition tables (`REFERENCING NEW TABLE AS`).

- PostgreSQL Global Development Group. *PL/pgSQL — SQL Procedural Language*. <https://www.postgresql.org/docs/16/plpgsql.html> — Consultado para la sintaxis de `sp_crear_pedido` (bloques `DECLARE/BEGIN/EXCEPTION`, manejo de `JSONB`, `RAISE EXCEPTION`).
- PostgreSQL Global Development Group. *Chapter 13. Concurrency Control*. <https://www.postgresql.org/docs/16/mvcc.html> — Base teórica para la sección de Transacciones y Concurrency: niveles de aislamiento (`READ COMMITTED`, `SERIALIZABLE`), `SELECT ... FOR UPDATE` y control de bloqueos.
- PostgreSQL Global Development Group. *13.2. Transaction Isolation*. <https://www.postgresql.org/docs/16/transaction-iso.html> — Detalle específico de los fenómenos de concurrencia (lectura no repetible, lost update) usados para diseñar el Escenario 3.
- PostgreSQL Global Development Group. *Using EXPLAIN*. <https://www.postgresql.org/docs/16/using-explain.html> — Referencia para interpretar los planes de ejecución (`Seq Scan`, `Bitmap Index Scan`, `Index Scan`) capturados en la justificación de índices.
- Elmasri, R., & Navathe, S. B. (2017). *Fundamentals of Database Systems* (7th ed.). Pearson. — Marco teórico general de modelo relacional, normalización (1FN, 2FN, 3FN, BCNF) y dependencias funcionales.
- Date, C. J. (2004). *An Introduction to Database Systems* (8th ed.). Addison-Wesley. — Referencia complementaria para las propiedades ACID y el concepto de claves candidatas usado en la demostración de BCNF sobre usuario.
- Apuntes y material de cátedra de Base de Datos 2 — UTN Facultad Regional Mendoza (Tecnatura Universitaria en Programación)

Ubicación:

Link a Repositorio: <https://github.com/marianoemir/TPI-BD-Food-Store>

Link a Video: <https://youtu.be/GBiR7vITjdQ>